⚠️ What is an Exception?

An **exception** is an **error** that occurs **during the execution of a program**, disrupting the normal flow. Python raises exceptions when something goes wrong.

Why Use Exception Handling?

- Prevent program crashes.
- Handle errors gracefully.
- Provide user-friendly error messages.

💥 Common Exceptions

Error	Cause
`ZeroDivisionError`	Dividing a number by zero
`FileNotFoundError`	Trying to open a file that doesn't exist
`TypeError`	Performing invalid operation on a data type
`ValueError`	Invalid value for a function/operation
`IndexError`	Accessing invalid index in a list
`Exception:`	Something went wrong.

Basic Syntax of Exception Handling

```
Syntax:1
try:
    # Code that may raise an exception
except ExceptionType(optional):
    # Code to handle the exception
Syntax:2
try:
    # Code
except Exception as e:
    print("An error occurred:", e)
```

In Java/C++:

```
try {
    // Code that might throw an exception
} catch (ExceptionType e) {
    // Handle exception
}
```

Example:

```
try:
  num = int(input("Enter a number: "))
  result = 10 / num
  print("Result:", result)
except:
  print("An error occurred.")
```

or

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print("Result:", result)
except ZeroDivisionError: #class name
    print("You can't divide by zero!")
except ValueError: #class name
    print("Please enter a valid number.")
```

Using `finally`

The `finally` block **always runs**, whether an exception occurs or not. The finally block in a try-except structure is used to **guarantee that certain code runs no matter what happens**, whether an exception was raised or not.

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print("Result:", result)
except ZeroDivisionError:
    print("You can't divide by zero!")
except ValueError:
    print("Please enter a valid number.")
finally:
    print("Execution finished. Thanks for using the program!")
```

`else` Block

The `else` block runs **if no exception** occurs in the `try` block. The `else` block **runs only if the `try` block succeeds** (i.e., no exceptions are raised). It's a way to separate the "happy path" (code that runs when everything goes well) from the `try` block.

```
try:
    # Code that might raise an exception
except SomeError:
    # Handle that error
else:
```

```
    # This runs only if try was successful (no exceptions)
finally:
    # This always runs, exception or not
```

Example:

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ZeroDivisionError:
    print("You can't divide by zero!")
except ValueError:
    print("Please enter a valid number.")
else:
    print("Result:", result)  # Only runs if no error happens
finally:
    print("Execution complete.")
```

📘 What is File Handling?

File Handling in Python allows you to **create, read, write, and delete files**. It is an essential skill for managing data in programs—whether you're working with text files, or data storage.

Python provides built-in functions and methods to handle files efficiently.

📂 Opening a File

You use the `open()` function to open a file:

```
file = open("filename.txt", mode)
```

- `filename.txt`: Name (and path) of the file.
- `mode`: The mode in which the file is opened (read, write, append, etc.).

Example 1: Opening a File in Read Mode (`"r"`)

```
# Read a file's content
file = open("example.txt", "r")
# Print the content
print(file.read())
# Close the file
file.close()
```

📁 1. Same Folder as Your Script

file = open("example.txt", "r")
Make sure example.txt is in the **same folder** as your .py file.

📁 2. Absolute Path

If your file is located somewhere else, provide the **full path**:
`file = open("C:/Users/YourName/Documents/example.txt", "r")`
Use double backslashes `\\` or forward slashes `/` on Windows.
On Mac/Linux, use `/Users/YourName/...`

📁 3. Relative Path

You can point to a file in another folder **relative** to your script:

```
file = open("data/example.txt", "r")  # Inside a folder named "data"
file = open("../example.txt", "r")    # One folder up
```

Example 2: Opening a File in Write Mode (`"w"`)

```
# Open the file for writing (creates or overwrites it)
file = open("example.txt", "w")

file.write("Hello from Python!")

file.close()
```

If the file doesn't exist, it is **created**. If it does exist, it is **overwritten**.

Example 3: Opening a File in Append Mode (`"a"`)

```
# Open the file to append new content at the end
file = open("example.txt", "a")

file.write("\nAdding more content.")

file.close()
```

This mode **adds to the file** without deleting existing content.

Example 4: Using `with` Statement (Recommended)

```
# Automatically closes the file after use
with open("example.txt", "r") as file:
     print(file.read())
```

`with` is safer and cleaner—it automatically handles closing the file, even if an error occurs.

Other Read Methods:

```
file.readline()      # Reads one line
file.readlines()     # Reads all lines into a list
file.read(5)         # Reads first 5 characters
```

File Modes:

Mode	Description
`'r'`	Read (default). File must exist.
`'w'`	Write. Creates a new file or overwrites existing.
`'a'`	Append. Adds to the end of the file.
`'x'`	Exclusive creation. Fails if the file exists.
`'b'`	Binary mode (e.g., images, PDFs).
`'t'`	Text mode (default for text files).
`'r+'`	Read and write. File must exist.
`'w+'`	Write and read. Overwrites file.
`'a+'`	Append and read. File is created if not exists.

Deleting a File

```
import os  # Import the OS module to work with file system

if os.path.exists("example.txt"):   # Check if file exists
    os.remove("example.txt")        # Delete the file
else:
    print("The file does not exist")
```

Explanation:

- `os` module: Helps interact with the operating system, like working with files and folders.
- `os.path.exists("filename")`: Checks if a file exists.
- `os.remove("filename")`: Deletes the file.

✅ Output Example:

If the file **exists**:

```
(example.txt is deleted silently)
```

If the file **does NOT exist**:

```
The file does not exist
```

⚠️ Without checking:

If you try this without checking first:

```
os.remove("example.txt")
```

It will raise:

```
FileNotFoundError: [Errno 2] No such file or directory: 'example.txt'
```

Python Libraries

A **library** in Python is a collection of pre-written code that provides reusable modules and functions to help developers build applications faster and more efficiently. Libraries in Python cover various domains such as **GUI development (like Tkinter, PyQt), data science (NumPy, Pandas, Matplotlib, Seaborn, Scipy,), machine learning (Scikit-learn, TensorFlow, PyTorch), web development (Flask, Django, FastAPI), automation (Selenium, PyAutoGUI, OpenCV, Pillow), game development (Pygame) and database management (SQLite3, SQLAlchemy).**

Tkinter

Tkinter is the built-in library in Python for creating graphical user interfaces (GUIs). It provides an easy-to-use interface to build desktop applications.

1. Installing Tkinter

Tkinter comes pre-installed with Python. You can check if Tkinter is installed using:

```
import tkinter
print(tkinter.TkVersion)
```

If not installed, install it using:

```
pip install tk
```

---

2. Creating a Basic Tkinter Window

```
import tkinter as tk
# Create the main window
root = tk.Tk()
# Set title and size
root.title("My First Tkinter App")
root.geometry("400x300")
# Run the application
root.mainloop()
```

Explanation

- `tk.Tk()`: Creates the main application window.
- `title()`: Sets the window title.
- `geometry()`: Defines the window size (`width x height`).
- `mainloop()`: Runs the application.

---

3. Tkinter Widgets

Tkinter provides various widgets to build an interface.

(a) `Label`

A **Label** is used to display text (or images) on the screen. It's read-only, meaning users can see it but not interact with it.

Syntax:

```
label = tk.Label(window_name, text="Your Text", options...)
```

Common options:

- `text="..."` → The text to display.
- `font=("FontName", size)` → Changes font style and size.
- `fg="color"` → Text color.
- `bg="color"` → Background color.
- `padx`, `pady` → Padding around the label.
- `width`, `height` → Size in text units (not pixels).

(b) Button

A **Button** lets the user perform an action when it's clicked.

Syntax:

```
button = tk.Button(window_name, text="Click Me", command=some_function,
options...)
```

Common options:

- `text="..."` → The label on the button.
- `command=function_name` → The function to call when clicked.
- `fg`, `bg`, `font`, `padx`, `pady`, `width`, `height` → Same as Label.

Note: `command` should pass only the function name (no parentheses), like `command=hello` instead of `command=hello()`.

Example: Label + Button

```
import tkinter as tk
# Create the main window
root = tk.Tk()
root.title("Label and Button Example")
root.geometry("300x200")
# Function that changes label text
def say_hello():
    message_label.config(text="Hello, Tkinter!")
# Create a Label
message_label = tk.Label(root, text="Click the button below!", font=("Arial",
14))
message_label.pack(pady=20)
# Create a Button
hello_button = tk.Button(root, text="Say Hello", command=say_hello,
font=("Arial", 12))
hello_button.pack()
# Run the app
root.mainloop()
```

What happens here:

- A label says “Click the button below!”.
- A button labeled **“Say Hello”**.
- When the button is clicked, the label changes to **“Hello, Tkinter!”**.

---

4. Tkinter Event Handling

Tkinter can handle user events like button clicks, key presses, etc.

(a) Handling Button Click

```
def on_click():
    print("You clicked the button!")

button = tk.Button(root, text="Click Me", command=on_click)
button.pack()
```

(b) Handling Key Press

```
def key_pressed(event):
    print(f"Key Pressed: {event.char}")

root.bind("<Key>", key_pressed)
```

(c) Handling Mouse Click

```
def mouse_click(event):
    print(f"Mouse Clicked at {event.x}, {event.y}")

root.bind("<Button-1>", mouse_click)
```

---

6. Tkinter Messagebox

Used for pop-up messages.

```
from tkinter import messagebox

def show_message():
    messagebox.showinfo("Info", "This is an info message.")

button = tk.Button(root, text="Show Message", command=show_message)
button.pack()
```

---

7. Tkinter Menu

Creating a menu bar.

```
menu_bar = tk.Menu(root)

file_menu = tk.Menu(menu_bar, tearoff=0)
file_menu.add_command(label="Open")
file_menu.add_command(label="Save")
file_menu.add_separator()
file_menu.add_command(label="Exit", command=root.quit)
```

```
menu_bar.add_cascade(label="File", menu=file_menu)
root.config(menu=menu_bar)
```

---

8. Tkinter Frames (Container for Widgets)

Frames are used to organize multiple widgets.

```
frame = tk.Frame(root, borderwidth=2, relief="solid")
frame.pack(pady=20)

label = tk.Label(frame, text="Inside Frame")
label.pack()
```

---

9. Tkinter Canvas (Drawing Shapes)

Canvas is used for drawing graphics.

```
canvas = tk.Canvas(root, width=200, height=200)
canvas.pack()

canvas.create_rectangle(50, 50, 150, 150, fill="blue")
```

---

10. Tkinter File Dialog (Open/Save Files)

```
from tkinter import filedialog

def open_file():
    file_path = filedialog.askopenfilename()
    print(f"Selected File: {file_path}")

button = tk.Button(root, text="Open File", command=open_file)
button.pack()
```

---

11. Tkinter Progress Bar

```
from tkinter import ttk

progress = ttk.Progressbar(root, orient="horizontal", length=200,
mode="determinate")
progress.pack()
progress.start(10)
```

---

12. Building a Simple Tkinter Application

A simple GUI with labels, buttons, and entry fields.

```
import tkinter as tk
from tkinter import messagebox

def greet():
    name = entry.get()
    messagebox.showinfo("Greeting", f"Hello, {name}!")

root = tk.Tk()
root.title("Simple App")
root.geometry("300x200")

tk.Label(root, text="Enter Your Name:").pack()
entry = tk.Entry(root)
entry.pack()

tk.Button(root, text="Greet", command=greet).pack()

root.mainloop()
```

What is `turtle` in Python?

- **`turtle`** is a **standard Python library** (comes built-in).
- It is mainly used for **graphics**, **drawing shapes**, **animations**, and **teaching programming** to beginners.
- It controls a **virtual pen ("turtle")** that draws on the screen.
- You give **commands** like move forward, turn left, change color, etc.

Basic Structure

```
import turtle
# Create a turtle object
pen = turtle.Turtle()
# Move the turtle
pen.forward(100)
pen.left(90)
pen.forward(100)
# Keep window open
turtle.done()
```

Important Turtle Commands

Command	What it does
`forward(x)` / `fd(x)`	Move turtle forward by `x` units
`backward(x)` / `bk(x)`	Move turtle backward by `x` units

Command	What it does
`right(angle)` / `rt(angle)`	Turn turtle clockwise by `angle` degrees
`left(angle)` / `lt(angle)`	Turn turtle counter-clockwise
`penup()`	Pick up the pen (no drawing)
`pendown()`	Put down the pen (start drawing)
`color(colorname)`	Change pen color
`fillcolor(colorname)`	Set fill color
`begin_fill()`	Start filling a shape
`end_fill()`	End filling the shape
`speed(speed)`	Set turtle speed (1 to 10 or "fastest")
`goto(x, y)`	Move to position `(x, y)`
`circle(radius)`	Draw a circle with given radius
`write(text, font=("Arial", size, "style"))`	Write text on screen

Drawing Example: Square

```
import turtle

pen = turtle.Turtle()

for _ in range(4):
    pen.forward(100)
    pen.left(90)

turtle.done()
```

✅ Draws a square!

Drawing Example: Circle

```
import turtle

pen = turtle.Turtle()

pen.circle(100)
```

```
turtle.done()
```

🧠 Why learn Turtle?

- It's fun! 🥳
- Helps you understand **loops**, **functions**, **graphics**, **coordinates**.
- Good starting point before learning **Pygame**, **OpenGL**, or big **game engines**.
